FUZE Software Quality Assurance Plan

Jerral Chua Fung Yong
Kataruka Mohika
Jethani Samarth
Sinha Armaan
Jain Sparsh
Faye Chan
Lai Yi

Team FUZE
College of Computing and Data Science, Nanyang Technological University

Submitted to—

Ajith Santhisenan

College of Computing and Data Science, Nanyang Technological University

Date: 15 September 2026
Revision 3.0

Signature Page

Prepared by: Samarth Jethani, Faye Chan, Armaan Sinha and Lai Yi
Date: 16th September 2026

Reviewed by 1: Samarth Jethani, QA Manager **Date:** 16th September 2026

Reviewed by 2: Sparsh Jain, Lead Developer **Date:** 16th September 2026

Reviewed by 3: Mohika Kataruka, Project Manager **Date:** 16th September 2026

Approved by: Ajith Santhisenan **Date:** _____

Document Change Record

Revision	Description of Change	Approved by	Date
1.0	Initial FUZE quality plan with DS-1–15 metrics, QA responsibilities and acceptance controls.	Pending	14 Sep 2026
2.0	Lab 2 review by Lai Yi on Samarth's 1.0 draft: client render fallback (DS-6), CSV and usability operating checks (DS-11, DS-15), 90-second warm-cache control beside the unchanged 130-second DS-5 gate, and vendor-source mapping from the 3 September minutes.	Pending	14 Sep 2026
3.0	Sparsh review of Lai Yi 2.0: replace DS-5 caching control (normalised physical properties only; stock/lifecycle/datasheet live); add DS-16 analytics with M4 seeded-corpus check, analysis tests, charting library, Lai Yi/Samarth accountabilities, and Lab 4 corpus risk.	Pending	15 Sep 2026

Contents

No.	Section	Page
1	Purpose and Scope	5
2	Reference Documents	5
3	Management	6
4	Documents	8
5	Standards, Practices, Conventions and Metrics	9
6	Software Reviews	10
7	Test	11
8	Problem Reporting and Corrective Action	12
9	Tools, Techniques and Methodologies	12
10	Media Control	12
11	Supplier Control	13
12	Record Collection, Maintenance, and Retention	13
13	Training	13
14	Risk Management	14
15	SQA Plan Change Procedure and History	14

1. Purpose and Scope

1.1. Purpose

This Software Quality Assurance (SQA) Plan defines the quality requirements, responsibilities, reviews and evidence needed to deliver FUZE, Team FUZE's system for whole-board constraint validation of electronic billsof materials (BOMs). It translates the proposal's DS-1 to DS-15 and the additional DS-16 analysis-surface checks into measurable checks throughout the 20 August to 25 November 2026 project.

The plan requires reproducible engine verdicts, traceable evidence, explicit missing-data handling and validated repair choices. Samarth leads verification and Faye leads validation. Milestone acceptance depends on demonstrated results and recorded sponsor sign-off; this plan does not record tests as completed or approvals as granted.

1.2. Scope

FUZEconvertsaplain-languagecircuit-boardbrieforuploadedBOMintoa graph of real components, checks compatibility and availability, proposes bounded repairs and presents a traceable final BOM. Users include hardware engineers, junior engineers and student design teams.

The plan covers the pure Python constraint engine, sourcing and normalisation, the cache of normalised part specifications, manufacturer datasheet ingestion, LangGraph orchestration, FastAPI interface, React client, CSV export, the analysissurface,accounts,persistenceanddeployment.TeamFUZEdevelops the application; JLCPCB, Mouser, the model provider and hosting services supply external dependencies. Schematic generation, routing, pin assignment, netlists, circuit simulation, manufacturing output and horizontal scaling are outside scope.

2. Reference Documents

- Project Proposal.pdf, Team FUZE: target specifications DS-1 to DS-16 and rule set (Tables 1–2, pp. 8–11); technologies and architecture (pp. 12–16).
- Project Proposal.pdf: phase and milestone plan (Tables 5–6, pp. 17–19); responsibilities (Table 7, pp. 19–20); deliverables (Table 8, pp. 20–22); sponsor communication (Table 10, pp. 24–26).
- CZ3002_Quality_Plan.doc: supplied SQA plan template and section structure. Its historical IEEE 730-2002 and ISO/IEC 90003:2004 references provide background only; no certification or standards compliance is asserted.
- 3 September 2026 in-class meeting minutes: caching structure to reduce live runtime from 3–4 minutes to about 90 seconds; fallback when the API fails to render; expansion of vendor sources beyond a single JLCPCB feed.
- Controlled project baselines: D1 Requirements Specification; D2 Interface Contract; D3 System Design Document; D6 Test Suite and Coverage Report; D7 Test Procedure Document; D8 Validation Evidence Pack; D10 Deployment Runbook; D11 User Guide; D12 Final Report.

The proposal supplies the acceptance baseline. Project documents are reviewed at their scheduled delivery gates; their listing here does not imply that they are already approved. Measurement procedures and operating controls below are the practices proposed by this SQA plan. Actions from the 3 September 2026 in-class lab (warm-cache runtime of about 90 seconds, explicit API/render fallback, and more than one component vendor) are operating controls. They do not relax any proposal DS threshold.

3. Management

3.1. Management Organisation

Quality management is shared by Samarth Jethani (Verification) and Faye Chan (Validation). Both report quality status and unresolved risks to Mohika Kataruka, Project Manager. Sparsh Jain is the technical design authority. Ajith Santhisenan is the named sponsor recipient in the proposal.

3.1.1. Project Management

Mohika coordinates approvals and is accountable for the following:

- Requirements baseline and sponsor acceptance of scope changes.
- Fourteen-week schedule, milestone readiness, budget and risk register.
- Trunk-based development with short-lived branches and recorded phase gates.
- Coordination with Sparsh on development tools, architecture and technical decisions.
- Team allocation and sufficient time for independent review and defect correction.
- Training coordination, sponsor communication and final handover.

3.1.2. Assurance Management

The QA managers report independently of the author of the work under review. No author approves their own change. Every change requires one non-author engineering review. Sparsh reviews engine or interface-contract changes. When QA personnel author a change, another qualified reviewer verifies it.

Samarth owns rule tests, the regression corpus, coverage gates, arithmetic cross-checks and adversarial tests. Faye owns browser acceptance tests, usability sessions and the sponsor evidence pack. Developers correct defects; Jerral Chua owns reproducible environments, fixture recording and release evidence.

3.2. Tasks

Assessments follow the DS identifiers and D1–D12 deliverables. Findings link to the affected requirement, build, owner and evidence in the project issue tracker.

3.2.1. Product Assessments

Each product is assessed against its applicable acceptance criteria:

- Engine and resolution policy: all ten rules, pure-function boundaries, determinism, origin, unchecked results, bounded repairs and legal model choices (DS-1–4, DS-7, DS-10).
- Data pipeline and interface: real part identifiers, stock sufficiency, normalised units, source preservation, cache of normalised physical properties (stock, lifecycle and datasheet links remain live), live and recorded latency, ordered streaming and recovery (DS-3, DS-5, DS-6, DS-9).
- Web application and export: conflict evidence, blank-or-stalled stream fallback, final post-repair CSV, first-time-user task completion, account access and secure sessions, and analysis views, including correct propagation of unchecked counts and correct withholding of a rate above threshold (DS-6, DS-11, DS-13, DS-15, DS-16).
- Deployment and evidence: health monitoring, cold starts, availability, secret handling, coverage, injection resistance, runbook reproducibility and release documentation (DS-8, DS-12–14; D6–D12).

3.2.2. Process Assessments

The following processes are checked at milestone reviews and during weekly QA review:

- Requirements and change control: trace DS-1–16 to tests and deliverables, record impact and written sponsor approval before implementing scope changes.
- Code and contract review: verify non-author approval, passing checks and Sparsh's review of engine or contract changes before merge.
- Test and defect management: preserve reproducible fixtures, expected outcomes, results, failure evidence, retests and regression tests.
- Configuration and release management: inspect dependency locks, migrations, protected credentials, release tags, rollback procedures and evidence retention.

3.3. Roles and Responsibilities

The following responsibilities apply to the project's two QA managers and supporting engineering team.

3.3.1. QAM

Samarth and Faye jointly maintain the quality plan and report release readiness. Their responsibilities are:

- Allocate the proposed 154 person-hours each across test design, execution, review and evidence preparation.
- Ensure access to the repository, CI results, recorded fixtures, browser environments and the demonstration deployment.

- Samarth: verify DS-1, DS-4, DS-8, DS-14 and DS-16; independently cross-check rule arithmetic and support Sparsh's DS-2, DS-3, DS-7 and DS-10 evidence.
- Faye: validate acceptance scenarios, DS-15 usability and the complete DS-1–16 evidence pack, coordinate M5 readiness.
- Escalate false passes, lost provenance, security defects and missed acceptance targets to Mohika, recommend withholding milestone acceptance until resolved.

3.3.2. Software Quality Personnel

Supporting software quality responsibilities are:

- Sparsh: technical review and accountability for DS-1–4, DS-7 and DS-10, M2 and M3.
- Armaan: backend, sourcing, orchestration, persistence and authentication; accountable for DS-5, DS-6, DS-9 and DS-13.
- Lai Yi: React client, CSV export, accessibility, user workflows and analysis views; accountable for DS-11, DS-15, DS-16 and the client side of DS-6, including pause/resume against sequence identifiers, explicit failure when the API or client renders nothing, and export-file checks in Excel and Google Sheets.
- Jerral: CI, deployment, fixtures, secrets, migrations, monitoring and runbook; accountable for DS-12 and infrastructure aspects of DS-13. All members provide evidence and resolve assigned findings.

4. Documents

4.1. Purpose

This section identifies the minimum documentation governing the requirements, development, verification, validation, and maintenance of software that falls within the scope of this software quality plan. Each document below shall be assessed (reviewed) by SQ personnel.

4.2. Minimum Document Requirements

QA reviews the following scheduled deliverables for completeness, correctness and traceability:

- D1 Requirements Specification (2 Sep): DS traceability, interview findings and freedom-to-operate memo. D2 Interface Contract (23 Sep): JSON Schema, types and ordering guarantees. D3 System Design Document (30 Sep): architecture, ten rule formulas, assumptions, repair policy and the analysis surface.
- D4 Engine Module (14 Oct): independent installation, interface and test instructions. D5 Complete Source Code (25 Nov): backend, frontend, infrastructure and README.
- D6 Test Suite and Coverage Report, D7 Test Procedures and D8 Validation Evidence Pack (18 Nov): regression, injection and seeded analytics corpora, repeatable setup, expected results, measured results and coverage.

- D9 Deployed System (11 Nov): deployment verification record. D10 Runbook, D11 User Guide and Training, and D12 Final Report and Demonstration (25 Nov): deployment and rollback, user tasks, limitations and results against every specification.

5. Standards, Practices, Conventions and Metrics

5.1. Purpose

This section highlights the standards, practices, quality requirements, and metrics to be applied to ensure a successful software quality program.

5.2. Software Quality Programme

Use typed interfaces, a frozen event schema, documented units and explicit source identifiers. Keep engine rules free of network, model and clock access. Store changes in short-lived branches, require non-author review and run automated checks before merge. Changes shall not be merged when mandatory CI checks or quality gates are failing.

The four qualities to be prioritised are correctness, reliability, efficiency and usability. Correctness requires all applicable rules and provenance to be accurate, with missing data reported as unchecked. Reliability requires deterministic verdicts, terminating repairs, secure state and recoverable streams. Efficiency is measured by BOM latency and event responsiveness. Usability requires first-time users to identify conflict causes and export a faithful BOM. Maintainability is supported by typed contracts, coverage gates and reproducible tests.

5.2.1. Standard Metrics

The acceptance metrics below retain the proposal's thresholds. QA records the numerator and denominator where relevant, test data, build, environment, procedure and evidence. Samarth consolidates automated results on each relevant CI run; Faye consolidates system and acceptance results at M4–M6. Failed or unexecuted checks remain visible. Where an operating control is tighter than a proposal gate (the 90-second warm-cache target versus the 130-second DS-5 live gate), both numbers are reported; only a failed proposal gate blocks milestone acceptance.

- DS-1 Rule coverage: at least 10 rules; each executes with zero network, model or system-clock access. Verify isolation and unit tests for voltage, current, thermal, interface role, pin budget, availability, temperature, footprint, energy and rail coverage. Owner: Sparsh; verifier: Samarth.
- DS-2 Determinism: byte-identical verdict sets across 100 consecutive runs of the same brief and cached part data. Freeze normalised inputs and record ordering/serialisation; separately exercise the brief-to-verdict path so normalisation drift is not hidden. Owner: Sparsh; verifier: Samarth.
- DS-3 Origin: 100% of verdicts include the deciding field name, verbatim value and a resolvable URL or named reference table; zero null sources. Trace from payload through normalisation to UI. Missing-field verdicts must identify absence and its source context without inventing a numeric value. Owner: Sparsh; verifier: Samarth.

- DS-4 No substituted defaults: at least 40 boards exercise missing required fields; every affected rule returns unchecked and names the field, with zero substituted defaults. Include sparse thermal data and unpowered components. Owner: Sparsh; verifier: Samarth.
- DS-5 End-to-end latency: validated BOM within 130 seconds with live data and 35 seconds with recorded data for boards of up to 12 slots. Measure submission to final validated BOM; preserve all timings, fixture versions, cache hit/miss and dependency errors. Owner: Armaan; validator: Faye. Caching control (3 September 2026 lab; does not replace this gate): without reuse, live runs take about 3 to 4 minutes, dominated by one language-model normalisation call per candidate part. Armaan maintains a cache of normalised part specifications, keyed by manufacturer part number and by a fingerprint of the normalisation prompt, so that a reworded instruction re-parses automatically rather than serving a stale interpretation. Cached fields are the component's physical properties, which do not change. Stock, lifecycle and datasheet links are read live on every run and are never served from cache, so no commercial value can go stale. Recorded distributor payloads held in the repository support offline regression replay and are not a runtime cache. The internal improvement target is about 90 seconds for a typical live 12-slot board on a warm cache. Cold-cache and cache-miss runs remain bounded by the 130-second gate. Report warm-cache, cold-cache and recorded-fixture times separately. Cached catalogue data must not be presented as a live stream event.
- DS-6 Streaming: first event within 2 seconds of submission, with zero gaps or duplicates, including pause/resume. Check sequence identifiers and client rendering against the server event log. If the FastAPI stream or the React client produces no events, a blank canvas, or a stalled last event, the client must show an explicit failure state with the error, the last sequence identifier and a retry/resume control. A silent blank screen is a defect. Owners: Armaan and Lai Yi; validator: Faye.
- DS-7 Bounded repair: no slot receives more than 3 repairs before user escalation; all runs terminate across 10 unsatisfiable boards. Exercise illegal and timed-out model choices and least-disruptive fallback. Owner: Sparsh; verifier: Samarth.
- DS-8 Test coverage: backend coverage at least 80%; constraint engine coverage at least 95%. Record executable-line coverage with pytest-cov, tested commit and any exclusions; fail CI below either threshold. Coverage does not replace rule-outcome tests. Owner: Samarth.
- DS-9 Sourceability: every BOM item maps to a real manufacturer part number and sufficient distributor stock, or is clearly flagged when unmet. Check quantities and stock against captured source evidence. Owner: Armaan; validator: Faye.
- DS-10 Model fencing: every proposed model repair is checked against engine-computed allowed options before application; zero unvalidated repairs accepted. Capture rejected choices and revalidation evidence. Owner: Sparsh; verifier: Samarth.
- DS-11 Export fidelity: CSV reflects the final post-repair board only, not an intermediate or failed run, and includes part number, quantity, price, stock

and datasheet URL. Compare every row to the client's final board state; open the file in Excel and Google Sheets; confirm the downloaded file matches the generated export. Owner: Lai Yi; validator: Faye.

- DS-12 Availability: at least 99% across the two-week demonstration period; cold start within 60 seconds and warm health responses within 500 ms. Jerral records probe interval, start/end times, all outages and response times; compute successful scheduled probes divided by all scheduled probes, treating missing samples as failures. Preserve the full two-week log. Owner: Jerral; validator: Faye.
- DS-13 Security: verify argon2id password hashing, secure session cookies and zero secrets committed to version control. Verify that authenticated users cannot access another user's projects or protected resources. Inspect configuration, negative authentication and authorisation tests and repository scan results; rotate any exposed credential before retest. Owners: Armaan and Jerral; verifier: Samarth.
- DS-14 Injection resistance: at least 25 injection briefs produce zero false pass verdicts. Compare outcomes to the independently defined expected rule results and test attempts to override validation or choose illegal repairs. Owner: Samarth; validator: Faye.
- DS-15 Usability: at least 8 of 10 first-time users independently run a design and identify a conflict cause within 10 minutes. The task uses the guided walkthrough, a brief that contains at least one conflict, and CSV export. Record task start/end, outcome, assistance and observations under anonymous participant IDs. Faye recruits the ten participants before M5; Lai Yi supplies the client build and the task script. Owner: Lai Yi; validator: Faye.
- DS-16 Validation analytics: across a seeded corpus of at least 20 completed runs, every figure on the analysis surface is reproducible by direct query against the stored findings and run events. Every displayed rate carries numerator, denominator and unchecked count. No rate is displayed where unchecked evaluations exceed 20% of the denominator. No derived score is persisted. Verify by removing a field a rule needs and confirming that the rule's evaluations move into the unchecked count rather than changing its failure rate. Owner: Lai Yi with Armaan; verifier: Samarth.

6. Software Reviews

6.1. Purpose

This section identifies the number and type of system/subsystem reviews and engineering peer reviews that will be supported by the SQ Personnel. The project milestone chart, and the SQ Personnel resource levels determine the reviews that are supported.

6.2. Minimum Software Reviews

Circulate milestone deliverables to the sponsor 48 hours before each review. QA checks traceability, test evidence, open defects, required attendees and exit

criteria. Mohika records decisions and actions with owners and due dates. A phase remains open until its exit criterion is demonstrated and written sponsor sign-off is recorded.

The following software reviews will be assessed by SQ:

- M1 Requirements and Project Plan Review — 2 Sep 2026: D1, testable DS-1–15, scope, resources and risks. Mohika leads; QA reviews acceptance feasibility.
- M2 Contract and Design Review — 23 Sep 2026: frozen D2 schema, types, provenance and ordering guarantees. Sparsh leads; both QA managers review. D3 design follow-up is due 30 Sep.
- M3 Engine and Test Review — 14 Oct 2026: independently runnable D4, all rules, arithmetic cross-checks, regression evidence and engine coverage. Sparsh and Samarth lead.
- M4 Integration Review — 4 Nov 2026: complete vertical slice against live data; sourceability, repair, streaming, warm-cache latency, client fallback, export evidence and analysis views against a seeded corpus. Armaan and Lai Yi demonstrate; Faye assesses M4 rather than M5, so that a thin or empty analysis surface is found with time to fix it rather than at validation.
- M5 Validation Review — 18 Nov 2026: D6–D8 and measured evidence for every DS criterion. Faye leads with Samarth. Any incomplete two-week availability observation remains explicitly open.
- M6 Release and Acceptance Review — 25 Nov 2026: complete availability window, final evidence, D5 and D10–D12, 20-minute demonstration, training and handover. Mohika coordinates sponsor sign-off. Peer code reviews occur throughout development.

7. Test

Samarth manages unit, property-based, contract, integration and adversarial verification. Faye manages end-to-end acceptance, usability and evidence consolidation. Each test records a DS identifier, setup, input data, expected outcome, execution steps, build and actual result. D7 must allow the sponsor to rerun acceptance independently. Live service tests complement recorded offline regression tests.

For each rule, test passing, failing, boundary and missing-data cases; independently calculate the expected arithmetic. Confirm current margin 0.15, footprint warning-only behaviour and rail-coverage unchecked behaviour. Recheck the entire board after every placement or replacement. Test authentication, project isolation, malformed uploads, CSV fidelity, stream recovery, cache hit versus miss timings, and the client’s explicit failure state when the API returns nothing or the stream stalls. Test the analysis queries against a seeded corpus with known counts, including a rule whose unchecked proportion exceeds the threshold, a user with no completed runs, and a user whose runs contain no repairs. Confirm that analysis returns only the authenticated engineer's runs and that abandoned and errored runs are excluded. Do not describe a board with unchecked results as fully validated. Release requires all DS criteria evidenced, regression checks passing and no unresolved false-pass, silent-blank-client or critical security defect.

8. Problem Reporting and Corrective Action

UseGitHubIssuesintheFUZEprojectrepositoryasthepproposedcentraldefect register, linked to the project board. Record ID, DS or deliverable, severity, environment, build, reproduction steps, expected/actual results, evidence, owner and target milestone. Workflow: Open → Triaged → In progress → Ready for retest → Closed; failed retests reopen the issue. Defects shall be classified as Critical, High, Medium or Low based on their impact on correctness, security, core functionality and release readiness.

QA triages findings weekly with the team and immediately escalates false passes, security exposure, data loss and release-blocking defects to Mohika. Developers supply the fix and regression test; a non-author retests before closure. Samarth and Faye report open counts by severity, ageing, recurrence, failed metrics and corrective-action status to Mohika weekly and for each milestone. Sponsor status reports and calls follow the proposal's bi-weekly Thursday schedule. Acceptance failures cannot be silently waived; any criterion change requires sponsor approval.

9. Tools, Techniques and Methodologies

The team will use the proposal's tool stack with the following quality controls and supporting techniques:

9.1. Software Quality Tools

- pytest and pytest-cov for unit/integration tests and coverage; Playwright for browser acceptance; the client charting library for DS-16 analysis views, pinned with the project dependencies; GitHub Actions for repeatable CI; GitHub repository and Issues for version control, reviews and defects. Property-based cases and hand calculations complement example tests.
- Use JSON Schema validation for the interface contract, recorded JLCPCB/Mouser fixtures for offline replay, the part-data cache with revision keys for warm-cache timing, HTTP/event logs for timing and sequence checks, and deployment health probes for availability. Use secret scanning and dependency checks in CI. Word and spreadsheets support procedures and metric summaries; Excel and Google Sheets verify CSV interoperability. Pin test-tool versions with the project dependencies.

10. Media Control

Keep source, schemas, tests, infrastructure and reviewable documentation under version control in the project repository. Store large evidence files in the sponsor-nominated shared drive, referenced by build and test ID. Restrict write access to team members and provide sponsor read access. Store credentials only in protected environments or secret settings, never in code, fixtures or exported evidence.

Jerral tags milestone and release baselines, records dependency versions and migration state, and archives CI evidence to the shared drive at each milestone so it survives CI expiry. Maintain a second copy of each release evidence bundle and

verify restore access before handover. QA records document revision, author, reviewer and associated milestone; superseded versions remain traceable.

11. Supplier Control

Supplier control applies to JLCPCB, Mouser, manufacturer datasheets, the model service and hosting/database providers. The 3 September 2026 lab action to move beyond a single JLCPCB feed is met by using JLCPCB for component parameters, stock and price, Mouser for lifecycle status and datasheet links, and manufacturer datasheets where catalogue data are incomplete. Adding a third distributor is a scope change and follows the change procedure below. Armaan verifies adapter schemas, units, identifiers, provenance and rate-limit/error handling. Record source and retrieval time outside the deterministic engine. Missing or conflicting data must remain explicit. Unexpected supplier API or schema changes shall fail explicitly rather than silently producing incomplete or incorrectly normalised data. Jerral controls service configuration and pinned dependencies. Test timeouts and outages with fixtures; accept model repairs only after engine validation. A supplier change requires an impact review, regression tests and updated evidence.

12. Record Collection, Maintenance, and Retention

Maintain electronic records that connect requirements, assessments, tests, findings, fixes and approvals. Samarth is custodian of verification evidence; Faye of validation and usability evidence; Jerral of release archives; Mohika of decisions and sign-offs. Store participant results under anonymous IDs and restrict any consent/contact records separately. Preserve evidence for one year after final handover, then review deletion or transfer with the sponsor.

The table below identifies the record types that will be collected, as well as the Record Custodian and Retention period:

Record Title	Record Custodian	Record Retention
Assessments, checklists and rule evidence	Samarth Jethani	One year after handover
Acceptance, usability and metric evidence	Faye Chan	One year after handover
Defects, fixes and retests	Samarth Jethani and Faye Chan	One year after handover

13. Training

Before undertaking assigned QA work, team members will complete project-specific walkthroughs and practical exercises. Samarth's SAP backend, API/integration testing and automation experience supports verification, it does not establish certification. Training topics and leads are:

- Rule arithmetic, units, missing data and provenance — Sparsh and Samarth.

- Risk assessment, change control and escalation — Mohika.
- Model fencing, injection tests and false-pass analysis — Samarth.
- Git reviews, CI, fixtures, secret handling, release and rollback — Jerral.
- JSON Schema contract, event ordering and pause/resume — Sparsh and Armaan.
- pytest, coverage interpretation and reproducible defect reporting — Samarth.
- Playwright, first-time-user sessions and acceptance evidence — Faye.
- Sponsor handover: brief writing, conflict evidence, BOM upload, datasheet attachment and export — Faye and Lai Yi, with D11 by 25 Nov. Record attendance, exercises and follow-up needs.

14. Risk Management

Mohika maintains the risk register. Both QA managers review it weekly with the team and at milestone gates; sponsor reviews follow the bi-weekly communication schedule. Each risk has an owner, likelihood, impact, trigger, mitigation, contingency and review date. Escalate any risk threatening correctness or a milestone as soon as it is identified.

Priority risks are missing or unreliable part data (Armaan: provenance and unchecked results); normalisation drift (Sparsh: frozen input/serialisation and 100-run tests); illegal or looping repairs (Samarth: adversarial tests, legal-set checks and cap); stream gaps or a blank/stalled client (Armaan and Lai Yi: sequence/recovery tests and an explicit error/retry state); a reworded normalisation prompt serving a stale physical interpretation (Armaan: cache keyed by part number and prompt fingerprint; stock, lifecycle and datasheet links are never cached); supplier limits/outages (Armaan: fixtures and explicit errors); hosting sleep or outage (Jerral: probes and local fallback); insufficient usability recruitment (Faye: recruit ten first-time participants before M5); and insufficient seeded run data for the analysis to demonstrate anything at Lab 4 (owner: Faye with Lai Yi; seeded corpus of at least twenty runs across several projects prepared and committed before M4; trigger: fewer than twenty completed runs in the demonstration account four weeks before M4). The 11 Nov deployment leaves only one week before M5: start the two-week availability observation by 4 Nov to close at M5, or record DS-12 as pending until 25 Nov and seek sponsor agreement on the gate timing without reducing the two-week requirement.

15. SQA Plan Change Procedure and History

Samarth and Faye maintain this plan. Proposed changes state the reason, affected DS criteria or deliverables, test impact, effort, schedule and risks. Sparsh reviews technical implications; Mohika coordinates approval and written sponsor consent for scope or acceptance changes. Update the revision record, link the decision and preserve the previous baseline. Re-run affected checks and notify the team of the new baseline. Revision1.0istheinitialpopulatedFUZEplan.Revision3.0applies
Sparsh’s 15 September Lab 2 comments on Lai Yi’s 2.0 draft: corrected DS-5 cache semantics and the matching scope wording, added DS-16 validation analytics against

the amended proposal baseline, and the matching product-assessment, deliverable, role, review, test, tool, training and risk updates. Approval remains pending.

Appendix A — Requirements Traceability Matrix

The following traceability matrix is included for convenience to provide a consolidated overview of each DS requirement, its ownership, verification or validation responsibility, associated evidence, deliverables and review milestones.

Requirement	Quality Requirement	Owner	Verifier / Validator	Test / Evidence	Related Deliverable	Review Milestone
DS-1	Rule coverage	Sparsh	Samarth	Unit tests, rule isolation tests	D6, D7	M3
DS-2	Determinism	Sparsh	Samarth	100-run deterministic test	D6, D8	M3
DS-3	Origin / provenance	Sparsh	Samarth	Provenance trace tests	D6, D8	M3/M4
DS-4	Missing-data handling	Sparsh	Samarth	40-board missing-data corpus	D6, D8	M3
DS-5	End-to-end latency	Armaan	Faye	Live + recorded timing results	D8	M4/M5
DS-6	Streaming	Armaan / Lai Yi	Faye	Sequence/recovery tests	D8	M4
DS-7	Bounded repair	Sparsh	Samarth	Unsatisfiable-board tests	D6, D8	M3
DS-8	Coverage	Samarth	Samarth / non-author reviewer	pytest-cov report	D6	M3
DS-9	Sourceability	Armaan	Faye	Stock/source evidence	D8	M4
DS-10	Model fencing	Sparsh	Samarth	Illegal repair rejection tests	D6, D8	M3
DS-11	Export fidelity	Lai Yi	Faye	CSV comparison	D8	M4

Requirement	Quality Requirement	Owner	Verifier / Validator	Test / Evidence	Related Deliverable	Review Milestone
DS-12	Availability	Jerral	Faye	Availability probe logs	D8/D10	M5/M6
DS-13	Security	Armaan / Jerral	Samarth	Auth + secret scan tests	D6/D8	M5
DS-14	Injection resistance	Samarth	Faye	25 adversarial briefs	D6/D8	M5
DS-15	Usability	Lai Yi	Faye	10-user usability test	D8	M5
DS-16	Validation analytics	Lai Yi / Armaan	Samarth	Seeded analytics/ query tests	D6/D8	M4/M5